ГБПОУ «Колледж автоматизации и информационных технологий № 20»

Методические указания к производственной практике

ПП.03 · ПМ.03 Ревьюирование программных продуктов
Неделя 1: модули 1–2 · специальность 09.02.07 · группа ИСПО332

Составитель: Беличенко А. А., 2026

Содержание 1. Как пользоваться методичкой 2. Подготовка рабочего места 3. Модуль 1. Теория: как исследовать чужой репозиторий 4. Модуль 1. Ход работы 5. Модуль 2. Теория: диаграммы потоков данных 6. Модуль 2. Ход работы 7. Оформление отчёта и сдача 8. Контрольные вопросы 9. Источники

1. Как пользоваться методичкой

Методичка объясняет, *как* выполнить задания недели 1; *что* и *к какому сроку* сдать — в Т3 на неделю 1. Для каждого модуля сначала идёт краткая теория (её спрашивают на защите), потом ход работы по шагам. Шаги выполняются по порядку: каждый опирается на результат предыдущего.

Репозиторий практики `pp03-ispo332` (ссылку даёт руководитель) — это ваш инструментарий. Сама работа ведётся в другом репозитории — в форке вашего проекта.

Папка в `pp03-ispo332`	Что там	Когда нужно
`modules/`	краткие инструкции к модулям 1 и 2 с командами	во время работы, как шпаргалка
`skeleton/`	шаблоны файлов, которые копируются в ваш проект (папка `practice/`)	один раз, в первый день
`tools/`	скрипты: установка шаблонов, статистика репозитория, объём кода, проверка готовности	М1 и перед каждой сдачей
`report-templates/`	шаблоны отчётов в Word для М1 и М2	в конце модуля
`examples/`	пример заполненного паспорта и реестра точек ввода	когда непонятно, как должно выглядеть
`docs/`	шпаргалка по Git, список программ, бланк документов практики	по необходимости

Условности: `ПРОЕКТ` — путь к папке вашего проекта, `PP03` — путь к папке `pp03-ispo332`. Команды вводятся в терминале: встроенном в VS Code (меню Terminal → New Terminal), PowerShell или Git Bash. В Windows вместо `python3` пишется `python`.

2. Подготовка рабочего места

Результат этапа (03.11, первые 2–3 часа): в вашем аккаунте лежит форк согласованного проекта с папкой `practice/`, а руководитель знает ссылку на него.

2.1 Программы

Программа	Зачем	Проверка
Git	контроль версий (в Windows вместе с ним ставится Git Bash)	`git --version`
VS Code	редактор	—
Python 3.8+	скрипты практики; при установке в Windows отметьте *Add python.exe to PATH*	`python --version`

Программа	Зачем	Проверка
Среда проекта	Node.js, PHP + Composer, JDK, .NET — что требует проект	`node -v`, `php -v` и т. д.
draw.io	диаграммы М2 (программа или расширение VS Code *Draw.io Integration*)	—

2.2 Настройка Git и SSH

```
git config --global user.name "Иванов Иван"
git config --global user.email "ivanov@example.com"
git config --global init.defaultBranch main
git config --global core.autocrlf true     # Windows; в Linux/macOS: input

ssh-keygen -t ed25519 -C "ivanov@example.com"   # Enter на все вопросы
cat ~/.ssh/id_ed25519.pub                         # этот текст — в Settings → SSH
keys
ssh -T git@github.com                             # должно поздороваться по логину
```

Файл `id_ed25519` без `.pub` — ваш закрытый ключ. Его нельзя никому отправлять и коммитить. На компьютерах колледжа, где SSH недоступен, используйте HTTPS-адрес репозитория и токен доступа вместо пароля.

2.3 Выбор проекта

Проверьте кандидата по списку, прежде чем идти к руководителю:

☐ есть клиент, сервер и БД (папки вроде `frontend/` + `backend/`, или фреймворк с шаблонами и моделями);

☐ во вкладке Commits — не менее 30 коммитов;

☐ есть README с инструкцией запуска и `.env.example` (или понятно, как настроить БД);

☐ в формах есть ввод данных: регистрация, создание записей, поиск (нужно для М2);

☐ стек вам хотя бы частично знаком.

Источники: свои проекты с УП и курсовых, поиск на GitHub: `topic:crud language:PHP stars:10..500` (вместо PHP — JavaScript, Python, Java, Kotlin, C#). Слишком крупные проекты (больше 50 тыс. строк) и учебные «todo-листы» на 200 строк не подходят.

2.4 Форк и шаблоны практики

1. На странице проекта нажмите **Fork** (GitHub) или создайте репозиторий импортом (GitFlic, GitLab). Свой старый проект форкать не нужно.
2. Клонируйте **свой** форк и скачайте репозиторий практики рядом:

```
cd ~/practice                                  # любая рабочая папка
git clone git@github.com:ВАШ_ЛОГИН/ПРОЕКТ.git
git clone <ссылка на pp03-ispo332>
```

1. Скопируйте шаблоны в проект (существующие файлы не перезаписываются):

```
# Windows (PowerShell), из папки pp03-ispo332:
powershell -ExecutionPolicy Bypass -File tools\install.ps1 ..\ПРОЕКТ
# Linux / macOS / Git Bash:
bash tools/install.sh ../ПРОЕКТ
```

1. Закоммитьте шаблоны в ветку первого модуля:

```
cd ../ПРОЕКТ
git switch -c practice/m01
git add practice .github
git commit -m "chore: add practice skeleton"
git push -u origin practice/m01
```

1. Заполните шапку `practice/README.md`, добавьте руководителя в репозиторий с правом чтения (Settings → Collaborators) и отправьте ему: ФИО, название проекта, ссылку на форк.

3. Модуль 1. Теория: как исследовать чужой репозиторий

Ревьюер не может оценить код, не понимая, что проект делает, из чего состоит и как развивался. Репозиторий отвечает на все три вопроса ещё до того, как вы прочитаете хотя бы одну функцию: структура каталогов показывает архитектуру, файлы зависимостей — стек, история Git — кто, когда и что менял.

3.1 Порядок чтения незнакомого проекта

1. **README** — что это за проект и как его запускают (по словам автора — потом проверим).
2. **Файлы-маркеры** в корне — на чём написан проект и от чего зависит (таблица ниже).
3. **Структура каталогов** — как код разложен по слоям.
4. **Точка входа** — файл, с которого начинается выполнение (`index.php`, `main.py`, `app.js`, `MainActivity.kt`, `Program.cs`), и маршруты.
5. **Конфигурация** — `.env.example`, `config/`, `docker-compose.yml`: какая БД, какие внешние сервисы.
6. **История** — как и кем развивался проект.

Файл в корне	Что означает	Где смотреть зависимости
`package.json`	JavaScript / TypeScript, Node.js	`dependencies`, `devDependencies`
`composer.json`	PHP (часто Laravel, Symfony)	`require`
`requirements.txt`, `pyproject.toml`	Python (Django, Flask, FastAPI)	список пакетов
`pom.xml`, `build.gradle(.kts)`	Java / Kotlin, Android	`dependencies`

Файл в корне	Что означает	Где смотреть зависимости
`*.csproj`, `*.sln`	C# / .NET	`PackageReference`
`pubspec.yaml`	Dart / Flutter	`dependencies`
`docker-compose.yml`	из каких сервисов состоит система (БД, кэш, веб-сервер)	`services`
`migrations/`, `database/`	структура БД и её история	файлы миграций

3.2 История Git как источник знаний

Команда	Что показывает	Какой вопрос закрывает
`git log --oneline --graph --all --decorate`	граф коммитов и веток	есть ли процесс (ветки, слияния) или всё в одну ветку
`git shortlog -sn --all`	число коммитов по авторам	один автор или команда
`git log --stat -5`	какие файлы менялись в последних коммитах	что сейчас разрабатывается
`git log --follow --oneline -- файл`	история одного файла, с учётом переименований	как развивался ключевой модуль
`git blame файл`	автор и коммит каждой строки	какие части дописаны позже, поверх старого кода
`git show <хеш>`	само изменение в коммите	зачем появилась строка
`git branch -a`, `git tag`	ветки и версии	есть ли релизы и параллельная разработка

Три понятия, которые пригодятся в выводах:

- **Горячая точка (hotspot)** — файл, который меняют чаще других. Часто это ядро логики или источник ошибок: там и начинают ревью.
- **Фактор автобуса (bus factor)** — сколько участников должны уйти, чтобы проект остановился. Если 90% коммитов одного автора — фактор равен 1.
- **Активность** — дата последнего коммита и распределение по месяцам. Проект без коммитов больше года, скорее всего, не соберётся со свежими версиями зависимостей.

3.3 Паспорт проекта

Паспорт проекта — короткий документ (2–4 страницы), по которому новый участник или ревьюер за 10 минут понимает, с чем имеет дело. Шаблон `practice/m01/passport.md` содержит 7 разделов: общие сведения, стек, структура каталогов, зависимости, история, запуск, вывод. Паспорт станет основой введения итогового отчёта, поэтому пишите его аккуратно. Пример заполнения — `examples/passport-example.md` в репозитории практики.

4. Модуль 1. Ход работы

Все записи делаете в `practice/m01/passport.md` вашего проекта. После каждого шага — коммит, например `docs: паспорт — структура каталогов`.

Шаг 1. Общие сведения (30 мин)

Прочитайте README и страницу репозитория. Заполните раздел 1: назначение своими словами (кто пользуется и для чего), ссылки на исходник и форк, лицензия (файл `LICENSE`; если его нет — так и напишите: формально такой код нельзя использовать в своих продуктах).

Шаг 2. Структура каталогов (1,5 ч)

```
git ls-files | head -60                          # файлы под контролем Git
tree -L 2 -I node_modules                        # Linux/macOS/Git Bash (если установлен
tree)
tree /F | more                                   # Windows cmd
```

В таблицу «Структура каталогов» внесите каждую папку верхнего уровня и ключевые файлы. Для каждой папки откройте 2–3 файла и напишите, что в ней *действительно* лежит, а не что подсказывает название. Найдите точку входа и файл маршрутов (`routes/web.php`, `urls.py`, `app.get(...)`, `@RestController`).

Шаг 3. Стек и зависимости (1,5 ч)

```
python PP03/tools/metrics.py ПРОЕКТ
```

Скрипт выведет таблицу «язык — файлов — строк кода — комментариев — пустых» (папки `node_modules`, `vendor`, `build` не считаются). Вставьте её в раздел 2 паспорта.

Зависимости: откройте файл-маркер, выберите 10–15 самых важных пакетов. Для незнакомого пакета откройте его страницу на npmjs.com, pypi.org или packagist.org и запишите назначение одной фразой. Отметьте сильно устаревшие версии — это первые кандидаты на уязвимости. СУБД ищите в `.env.example` (`DB_CONNECTION`, `DATABASE_URL`), `docker-compose.yml`, настройках ORM.

Шаг 4. История репозитория (2,5 ч)

```
python PP03/tools/repo_stats.py ПРОЕКТ --out ПРОЕКТ/practice/m01/repo-stats.md
cd ПРОЕКТ
git log --oneline --graph --all --decorate -n 40      # скриншот → в отчёт
git log --follow --oneline -- путь/к/ключевому/файлу
git blame путь/к/ключевому/файлу | head -40
```

Скрипт соберёт в `repo-stats.md`: число коммитов и период, авторов, ветки и теги, 10 самых часто изменяемых файлов, активность по месяцам. Цифры — не цель, цель — выводы в разделе 5 паспорта. Ответьте письменно:

- проект живой или заброшен (первая и последняя дата, активность)?
- какой фактор автобуса?
- какие файлы — горячие точки и почему (откройте их и посмотрите `git log` по ним)?
- что показал `git blame` по ключевому файлу: код написан сразу или обрастал заплатками?
- использовались ли ветки и Pull Request или всё коммитилось в main?

Шаг 5. Запуск (2–3 ч)

Запустите проект по README и записывайте в раздел 6 каждую команду, которая сработала.

Проблема	Решение
нет файла `.env`	скопировать из `.env.example` и заполнить локальными данными; `.env` не коммитить
не ставятся зависимости	посмотреть требуемую версию Node / PHP / Python (`engines` в package.json, `require.php` в composer.json)
пустая БД	выполнить миграции и сиды (`php artisan migrate --seed`, `python manage.py migrate`)
порт занят	сменить порт в `.env` или параметрах запуска

Не запускается за 2 часа — не тратьте день, обратитесь к руководителю: возможно, проект придётся сменить. Если вам пришлось дописать шаги запуска — исправьте README и закоммитьте: `docs: дополнена инструкция запуска`. Это уже первая находка ревьюера.

Шаг 6. Вывод, отчёт, сдача (2 ч)

Раздел 7 паспорта — 5–7 предложений: насколько проект зрелый и где вероятны проблемы, с опорой на факты из шагов 2–5. Далее — отчёт по шаблону (раздел 7 методички) и проверка:

```
python PP03/tools/check_practice.py ПРОЕКТ --module 1
```

5. Модуль 2. Теория: диаграммы потоков данных

DFD (Data Flow Diagram, диаграмма потоков данных) показывает, откуда данные попадают в систему, какие процессы их преобразуют и где они хранятся. В отличие от блок-схемы, DFD не показывает порядок действий, условия и циклы — только движение данных. Для ревьюера это карта: по ней видно, где в систему входят непроверенные данные и где они оседают.

5.1 Элементы и нотации

На практике используется нотация Гейна–Сарсона. Нотация Йордона–Де Марко тоже принимается, но смешивать их на одной диаграмме нельзя.

Элемент	Гейн–Сарсон	Йордон–Де Марко	Что это в веб-проекте	Как называть
Внешняя сущность	прямоугольник с тенью	прямоугольник	пользователь, администратор, платёжный сервис, почтовый сервер	существительное: «Читатель»
Процесс	скруглённый прямоугольник с номером сверху	круг	контроллер, обработчик маршрута, сервис	глагол + объект: «Оформить выдачу»
Хранилище	открытый справа прямоугольник с номером D1, D2…	две параллельные линии	таблица БД, файл, кэш, localStorage	существительное во мн. числе: «Выдачи»
Поток данных	стрелка с подписью	стрелка с подписью	HTTP-запрос, ответ, SQL-запрос	что передаётся: «логин, пароль»

5.2 Уровни детализации

- **Контекстная диаграмма (уровень 0)** — вся система одним процессом «0», вокруг все внешние сущности и потоки между ними и системой. Хранилищ на ней нет.
- **Уровень 1** — процесс «0» раскрыт на 4–8 процессов (1, 2, 3…), появляются хранилища.
- **Уровень 2** — при необходимости один процесс раскрывается дальше (2.1, 2.2…). На практике не обязателен.

Балансировка: внешние потоки контекстной диаграммы должны все встретиться на уровне 1 — ни потерянных, ни новых.

5.3 Правила и типичные ошибки

- У каждой стрелки есть подпись — что именно передаётся. Стрелка без подписи — ошибка.
- Данные не идут напрямую между сущностью и хранилищем, между двумя хранилищами или двумя сущностями — только через процесс.
- **«Чёрная дыра»** — процесс, у которого есть входы, но нет выходов.
- **«Чудо»** — процесс, у которого есть выходы без входов.
- **«Серая дыра»** — выход нельзя получить из входов (процесс выдаёт «список книг», но ниоткуда их не читает).
- Хранилище, в которое только пишут или из которого только читают, — повод проверить, не забыли ли вы процесс.

5.4 Пример: DFD уровня 1 для «Электронной библиотеки»

Пример упрощён до трёх процессов; в вашей диаграмме их будет 4–8.

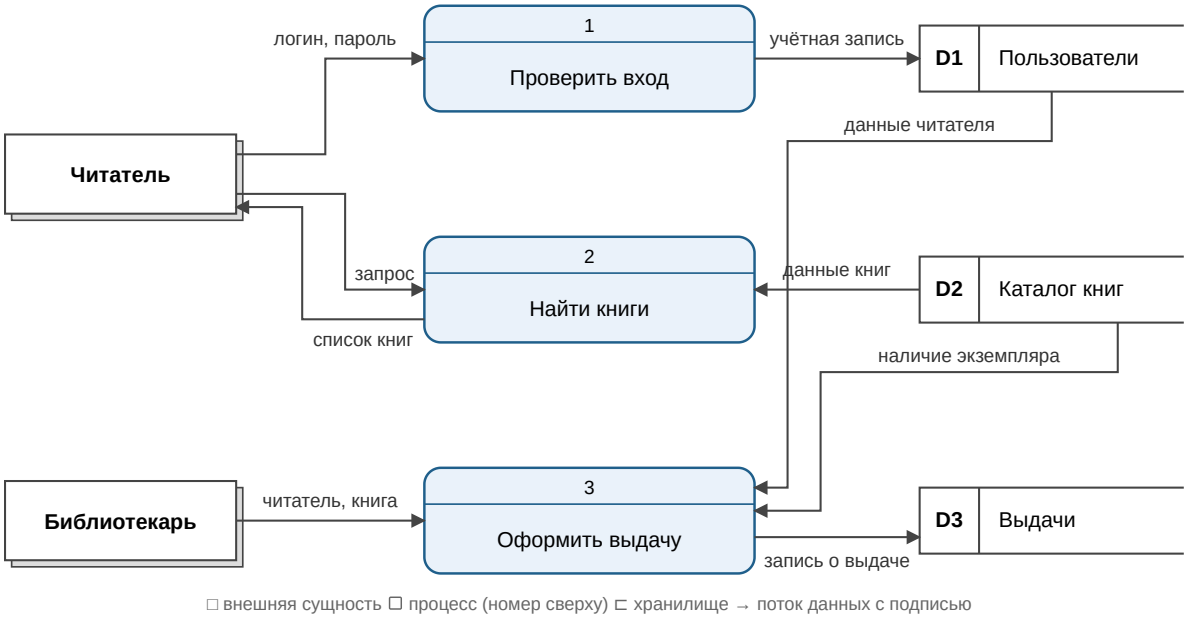


Рисунок 1 — DFD уровня 1 «Электронная библиотека» (нотация Гейна–Сарсона)

Как читать: читатель передаёт логин и пароль процессу 1, который сверяет их с хранилищем D1 «Пользователи»; поисковый запрос обрабатывает процесс 2, читая D2 «Каталог книг»; выдачу оформляет библиотекарь через процесс 3, который читает D1 и D2 и записывает результат в D3 «Выдачи». Ни одна сущность не обращается к хранилищу напрямую.

5.5 От диаграммы к коду: трассировка и точки ввода

Трассировка — проход одного сценария по коду от кнопки до записи в БД и обратно, с указанием `файл:строка` на каждом шаге. Она проверяет диаграмму: если в коде данные идут не так, как нарисовано, — исправляется диаграмма.

Точка ввода — любое место, где данные извне попадают в систему: поле формы, параметр URL (`?id=5`), тело API-запроса, загруженный файл, cookie, заголовок. На DFD это стрелки от внешних сущностей к процессам. Именно через них происходят самые частые атаки — инъекции (SQL, XSS) входят в список OWASP Top 10. Правило ревьюера: данным от клиента нельзя доверять, даже если форма их проверяет — запрос можно отправить в обход формы. Поэтому в реестре для каждой точки отмечается отдельно проверка на клиенте и на сервере.

HAR (HTTP Archive) — файл, в который браузер сохраняет все запросы страницы с заголовками и телами. Это доказательство того, какие данные реально идут по сети. В нём есть cookies и токены, поэтому перед публикацией их нужно заменить на `***`.

6. Модуль 2. Ход работы

Новая ветка: `git switch main && git pull && git switch -c practice/m02`. Все файлы — в `practice/m02/`.

Шаг 1. Элементы DFD (2 ч)

Заполните таблицу «Элементы DFD» в `trace.md`. Где искать:

Элемент	Где искать в коде
Внешние сущности	роли пользователей (`role`, `is_admin`, middleware авторизации), внешние API (HTTP-клиенты, SDK платёжных и почтовых сервисов)
Процессы	маршруты и контроллеры: `routes/`, `urls.py`, `@GetMapping`, `app.post(...)` — группируйте по смыслу, а не по одному маршруту на процесс
Хранилища	модели ORM, миграции, `CREATE TABLE`, работа с файлами (`storage/`, `uploads/`), кэш, `localStorage` на клиенте

Шаг 2. Диаграммы в draw.io (4 ч)

1. Создайте `practice/m02/dfd.drawio`. В draw.io включите библиотеку фигур: More Shapes → *Software* или используйте простые фигуры по образцу рисунка 1.
2. **Лист 1 — контекстная диаграмма**: система одним процессом «0 Название системы», вокруг внешние сущности, подписанные потоки.
3. **Лист 2 — DFD уровня 1**: 4–8 процессов с номерами, хранилища D1…Dn, потоки.
4. Проверьте по правилам из п. 5.3: подписи у всех стрелок, нет потоков «сущность → хранилище», нет чёрных дыр и чудес, балансировка с контекстной диаграммой.
5. Экспорт каждого листа: File → Export as → PNG → `dfd-0.png`, `dfd-1.png`. Закоммитьте.

Шаг 3. Трассировка двух сценариев (4 ч)

Выберите два сценария, где пользователь **вводит** данные (регистрация, создание записи, поиск, загрузка файла). Пройдите по коду:

1. Найдите форму на клиенте — поиск по тексту кнопки: Ctrl+Shift+F в VS Code.
2. Найдите, куда уходит запрос: `fetch`, `axios`, `action="..."`, Retrofit-интерфейс.
3. Найдите обработчик этого маршрута на сервере.
4. Найдите запрос к БД: ORM-метод или SQL.
5. Найдите, что возвращается клиенту.

Каждый шаг — строка таблицы в `trace.md` с `файл:строка`. Если в коде нашлось то, чего нет на диаграмме (например, запись в лог или отправка письма), — дополните диаграмму.

Шаг 4. Перехват запросов (2 ч)

1. Войдите в приложение под **тестовым** пользователем. Откройте DevTools (F12) → вкладка **Network**, включите *Preserve log*.
2. Выполните сценарий 1. Найдите запрос (фильтр *Fetch/XHR* или *Doc*), посмотрите вкладки Headers, Payload, Response. Скриншот — в отчёт.
3. Правый клик по списку запросов → *Save all as HAR* → `practice/m02/scenario1.har`.
4. Откройте HAR в VS Code, найдите `password`, `token`, `Authorization`, `Cookie` и замените значения на `***`. Только после этого коммитьте.
5. Сверьте: метод, URL и поля запроса совпадают с тем, что вы нашли в коде на шаге 3? Расхождения запишите в раздел «Расхождения».

Мобильный проект: используйте App Inspection → Network Inspector в Android Studio или логирование запросов на сервере.

Шаг 5. Реестр точек ввода (3 ч)

В `input-points.md` перечислите **все** точки ввода — формы, параметры URL, тела API-запросов, загрузку файлов. Для каждой:

- поля и их тип;
- проверка на клиенте: атрибуты `required`, `type="email"`, `maxlength`, проверки в JS — да / нет;
- проверка на сервере: валидатор фреймворка (`$request->validate`, `Form.is_valid()`, `@Valid`, схема Joi/Zod) — да / нет;
- риск: что будет, если отправить пустое, слишком длинное или специально составленное значение.

Пример заполнения — `examples/input-points-example.md`. «Нигде» — тоже ответ, и именно такие строки станут задачами в модулях 10, 11 и 13.

Шаг 6. Вывод, отчёт, сдача (1 ч)

Вывод — 5–7 предложений: что совпало и не совпало с ожиданиями, сколько точек ввода, сколько из них без серверной проверки, какие самые рискованные. Отчёт по шаблону, проверка `python PP03/tools/check_practice.py ПРОЕКТ --module 2`, Pull Request.

7. Оформление отчёта и сдача

Отчёт по каждому модулю оформляется по единому образцу колледжа. Готовые шаблоны — `report-templates/report-M01.docx` и `report-M02.docx`: в них уже заполнены титульный лист (кроме ФИО), оглавление, цель и задачи модуля, а в остальных разделах стоят серые подсказки, что писать. Сохраните копию шаблона под именем `ISP0332_Familiya_M01.docx` (ваша фамилия латиницей) и замените подсказки своим текстом.

Раздел	Что в нём
Титульный лист	вид практики, ФИО, профессиональный модуль, номер и название модуля практики, курс, группа, место, сроки, объём часов
Оглавление	обновляется автоматически: правый клик → «Обновить поле» → «Обновить целиком»
Введение	2–3 абзаца: о практике, о модуле, какой проект исследуется
Краткое описание проекта	объект исследования: назначение, тип приложения, ссылки на исходник и форк
Цель и задачи	уже заполнено в шаблоне
Методы и инструменты анализа	какие методы применяли и какими программами
Ход работы	по каждому заданию: что сделано, команды, таблицы, скриншоты
Заключение	результаты (созданные файлы со ссылками) и выводы — не менее 5 предложений с опорой на факты
Список используемых источников	с URL и датой обращения
Приложения	большие таблицы и дополнительные скриншоты

Оформление: Times New Roman 14 пт, интервал 1,5, заголовки разделов 16 пт полужирные, каждый раздел с новой страницы, рисунки по центру с подписью «Рисунок 1 — Граф коммитов проекта». Обычный объём — 8–12 страниц вместе с титульным листом.

Сдача модуля:

1. Отчёт — в `practice/mNN/`, коммит.
2. `python PP03/tools/check_practice.py ПРОЕКТ --module N` — без ошибок.
3. Pull Request из ветки `practice/mNN` в основную, заполнить шаблон описания, влить.
4. Отметить модуль `[x]` в `practice/README.md`.

8. Контрольные вопросы

Вопросы входят в защиту практики. Отвечайте на примерах из своего проекта.

Модуль 1

1. Чем форк отличается от клона? Зачем на практике нужен форк?
2. По каким файлам в корне репозитория можно определить стек проекта? Приведите три примера.
3. Что показывают `git log --graph`, `git shortlog -sn` и `git blame`? Какие выводы о вашем проекте вы сделали с их помощью?
4. Что такое горячая точка репозитория и почему с неё начинают ревью?
5. Что такое фактор автобуса? Какой он у вашего проекта?

6. Почему файл `.env` не хранят в репозитории? Что делать, если секрет уже попал в историю?
7. Как по репозиторию понять, что проект заброшен?

Модуль 2

1. Чем DFD отличается от блок-схемы алгоритма?
2. Назовите элементы DFD и их обозначения в нотации Гейна–Сарсона.
3. Чем контекстная диаграмма отличается от диаграммы уровня 1? Что такое балансировка?
4. Почему поток не может идти напрямую из хранилища во внешнюю сущность?
5. Что такое «чёрная дыра», «чудо» и «серая дыра» на DFD?
6. Что такое точка ввода? Почему клиентской проверки данных недостаточно?
7. Что такое HAR-файл и почему его опасно публиковать без обработки?

9. Источники

1. Chacon S., Straub B. Pro Git. 2-е изд. — URL: https://git-scm.com/book/ru/v2 (главы 2 «Основы Git» и 7.10 «Обнаружение ошибок с помощью Git»).
2. Gane C., Sarson T. Structured Systems Analysis: Tools and Techniques. — Prentice Hall, 1979.
3. Вендров А. М. Проектирование программного обеспечения экономических информационных систем. — М.: Финансы и статистика, 2006 (глава о структурном анализе и DFD).
4. OWASP Top 10. — URL: https://owasp.org/www-project-top-ten/
5. Chrome DevTools: Network features reference. — URL: https://developer.chrome.com/docs/devtools/network/reference
6. draw.io: документация. — URL: https://www.drawio.com/doc/